Universidad Politécnica de Atlauta

Alumno: David Fernando Rosales Huesca

Maestro: Garcia Lara Javier

Tema: Segundo entregable de la meteria Fundamentos de Inteligencia Artificial

Fecha: 18/10/2026

Modelado de problemas reales como problemas de búsqueda en grafos

Modelado y código: formulación de estado, acciones, costo y meta; representación como grafo dirigido y ponderado; implementación en Python con NetworkX; análisis de complejidad.

1. Introducción

Un problema de búsqueda se formula describiendo cuatro elementos: el **estado** (qué configuración del mundo se está considerando), las **acciones** (qué transiciones están permitidas en cada estado), el **costo** (cuánto cuesta ejecutar cada acción) y la **meta** (qué condición debe cumplir el estado final). Al aplicar sistemáticamente las acciones desde el estado inicial se obtiene el **espacio de estados**, que se representa de forma natural como un grafo dirigido y ponderado: cada nodo es un estado, cada arista dirigida es una acción y el peso de la arista es su costo.

En este trabajo se modelan dos problemas reales:

1. **Ruta de reparto:** un repartidor que debe atender a varios clientes minimizando los kilómetros recorridos.
2. **Dosificación de líquidos:** medir una cantidad exacta de líquido con dos recipientes de capacidad conocida, minimizando los litros manipulados.

Para ambos se programó en Python un generador de estados que construye el grafo con NetworkX, se ejecutaron dos algoritmos de búsqueda y se analizó la complejidad.

2. Fundamentos

2.1 Formulación de un problema de búsqueda

Elemento	Descripción
Estado inicial	Configuración desde la cual comienza la búsqueda.
Acciones(s)	Conjunto de acciones aplicables en el estado s.
Modelo de transición	Estado que resulta de aplicar una acción a un estado.
Costo de acción	Valor numérico positivo asociado a cada transición.
Prueba de meta	Condición que identifica los estados solución.

Una solución es una secuencia de acciones que lleva del estado inicial a un estado meta. Es óptima si su costo total es mínimo.

2.2 Espacio de estados como grafo

Se define el grafo G = (V, E, w) donde V es el conjunto de estados alcanzables, E es el conjunto de transiciones (aristas dirigidas) y w asigna a cada arista el costo de la acción. Buscar una solución equivale a encontrar un camino desde el nodo inicial hasta algún nodo meta; buscar la solución óptima equivale a encontrar el camino de menor peso total.

2.3 Algoritmos utilizados

- **Búsqueda en anchura (BFS):** expande los nodos por niveles. Garantiza la solución con menos acciones, pero no necesariamente la de menor costo.
- **Búsqueda de costo uniforme (UCS, equivalente a Dijkstra):** expande siempre el nodo con menor costo acumulado usando una cola de prioridad. Garantiza la solución de costo mínimo cuando todos los costos son positivos.

Los resultados de UCS se contrastaron con el algoritmo de Dijkstra incluido en NetworkX como verificación independiente.

3. Problema 1: ruta de reparto

3.1 Descripción

Un repartidor sale del depósito en Ixtapaluca y debe entregar un paquete en tres localidades: Chalco, Los Reyes y Texcoco. No necesita regresar al depósito. Se busca el recorrido de menor distancia. Las distancias son valores ilustrativos definidos para el ejercicio.

3.2 Red de carreteras (grafo dirigido)

Origen	Destino	Kilómetros
Ixtapaluca	Chalco	14
Chalco	Ixtapaluca	14
Ixtapaluca	Los Reyes	10
Los Reyes	Ixtapaluca	10
Los Reyes	Texcoco	18
Texcoco	Los Reyes	18
Ixtapaluca	Texcoco	28
Texcoco	Ixtapaluca	30
Chalco	Los Reyes	17 (un solo sentido)

La asimetría de las distancias entre Ixtapaluca y Texcoco, y el tramo de un solo sentido entre Chalco y Los Reyes, hacen que el grafo sea realmente dirigido y ponderado.

3.3 Formulación

Elemento	Definición
Estado	Par (ubicación, entregados): la localidad donde se encuentra el repartidor y el conjunto de clientes ya atendidos.
Estado inicial	(Ixtapaluca, conjunto vacío).
Acciones	Desplazarse por una carretera hacia una localidad vecina. Si la localidad es un cliente, la entrega se registra al llegar.
Costo	Kilómetros de la carretera recorrida.
Meta	Los tres clientes han sido atendidos.

3.4 Tamaño del espacio de estados

Con n = 4 localidades y k = 3 clientes, el límite superior de estados es n · 2^k = 32. El generador encontró **20 estados alcanzables**, porque muchas combinaciones son imposibles (por ejemplo, estar en Chalco sin haber entregado en Chalco). Hay 4 estados meta, uno por cada localidad donde puede terminar el repartidor.

4. Problema 2: dosificación de líquidos

4.1 Descripción

En un proceso de dosificación se dispone de un recipiente A de 3 litros y un recipiente B de 5 litros, sin marcas de graduación. Se necesita obtener exactamente 4 litros en el recipiente B. Cada operación tiene un costo igual a los litros de líquido que se agregan, se desechan o se trasvasan.

4.2 Formulación

Elemento	Definición
Estado	Par (a, b): litros en el recipiente A y en el recipiente B, con $0 \leq a \leq 3$ y $0 \leq b \leq 5$.
Estado inicial	(0, 0).
Acciones	Llenar A, llenar B, vaciar A, vaciar B, verter A en B, verter B en A. Se descartan las acciones que no modifican el estado.
Costo	Litros agregados, desechados o trasvasados en la acción.
Meta	b = 4.

4.3 Tamaño del espacio de estados

El límite superior es (3 + 1) · (5 + 1) = 24 estados. Como 3 y 5 son coprimos, solo son alcanzables los estados en los que al menos un recipiente está vacío o lleno, es decir 2 · (3 + 5) = 16 estados, que coincide con lo que obtuvo el generador. Los estados meta son (0, 4) y (3, 4).

5. Diseño de la implementación

El programa `modelado_busqueda.py` se organiza en cuatro partes:

1. **Clase base `ProblemaBusqueda`:** define la interfaz común con los métodos `estado_inicial`, `acciones`, `es_meta` y `etiqueta`. Cada problema hereda de esta clase, de modo que el generador y los algoritmos funcionan con cualquier problema sin modificarse.
2. **Clases `RutaReparto` y `DosificacionLiquidos`:** implementan la formulación de cada problema. El método `acciones` devuelve tuplas (nombre de la acción, estado resultante, costo).
3. **Función `generar_grafo`:** es el generador de estados. Parte del estado inicial, explora en anchura con una cola y un conjunto de visitados, y por cada transición agrega a un `nx.DiGraph` el nodo destino y una arista con los atributos `weight` y `accion`. Si dos acciones llevan del mismo estado al mismo estado se conserva la de menor costo.
4. **Algoritmos y diagramas:** `busqueda_anchura`, `costo_uniforme` (con `heapq`), una verificación con `nx.single_source_dijkstra_path_length` y la función `dibujar_grafo`, que genera los diagramas en PNG resaltando en verde el estado inicial, en dorado los estados meta y en rojo el camino óptimo.

6. Resultados

6.1 Métricas del grafo

Métrica	Ruta de reparto	Dosificación de líquidos
Estados alcanzables (V)	20	16
Transiciones (E)	48	58
Estados meta	4	2
Factor de ramificación medio (E / V)	2.40	3.62
Tiempo de generación	0.45 ms	0.17 ms

6.2 Comparación de algoritmos

Resultado	Ruta de reparto	Dosificación de líquidos
BFS: acciones / costo / nodos expandidos	3 / 49 / 13	6 / 19 / 14
UCS: acciones / costo / nodos expandidos	3 / 49 / 12	6 / 19 / 14

Resultado	Ruta de reparto	Dosificación de líquidos
Costo mínimo según Dijkstra de NetworkX	49	19

En ambos problemas el costo obtenido por UCS coincide con el de NetworkX, lo que valida la implementación.

6.3 Soluciones óptimas

Ruta de reparto (49 km): Ixtapaluca → Chalco (14 km) → Los Reyes (17 km) → Texcoco (18 km).

Dosificación de líquidos (19 litros manipulados):

Paso	Acción	Estado resultante	Costo
1	Llenar B	(0, 5)	5
2	Verter B en A	(3, 2)	3
3	Vaciar A	(0, 2)	3
4	Verter B en A	(2, 0)	2
5	Llenar B	(2, 5)	5
6	Verter B en A	(3, 4)	1

6.4 Diagramas del grafo

Los diagramas se entregan como imágenes junto con el código: `grafo_ruta_reparto.png` y `grafo_dosificacion.png`. En el primero los nodos se distribuyen en columnas según el número de acciones desde el estado inicial; en el segundo los nodos se ubican en una cuadrícula cuyos ejes son los litros de A y de B. Las aristas rojas marcan el camino de costo mínimo.

7. Análisis de complejidad

Se usan las siguientes variables: V es el número de estados alcanzables, E el número de transiciones, b el factor de ramificación, d la profundidad de la solución menos profunda, C* el costo de la solución óptima y ε el menor costo de una acción.

7.1 Generación del grafo

Cada estado se expande una sola vez gracias al conjunto de visitados y por cada expansión se generan como máximo b sucesores. El tiempo es **O(V · b) = O(V + E)**, suponiendo operaciones de hash de costo constante. El espacio es **O(V + E)**, porque se almacena el grafo completo.

7.2 Búsqueda en anchura

Sobre el grafo ya construido, BFS visita cada nodo y cada arista a lo sumo una vez: tiempo **O(V + E)** y espacio **O(V)** por la cola y el diccionario de padres. Sobre el árbol de búsqueda sin

detección de repetidos, el peor caso sería **O(b^d)**; la detección de estados repetidos es lo que acota el costo por V.

7.3 Búsqueda de costo uniforme

Con una cola de prioridad binaria, cada arista puede provocar una inserción de costo O(log V), por lo que el tiempo es **O((V + E) log V)** y el espacio **O(V)**. Sin detección de repetidos, la cota teórica es *O(b^(1 + ⌊C/ε⌋))*. UCS es completo y óptimo si ε > 0, condición que se cumple en ambos problemas porque se descartaron las acciones de costo cero.

7.4 Complejidad de cada problema

Aspecto	Ruta de reparto	Dosificación de líquidos
Cota de V	n · 2^k (n localidades, k clientes)	(A + 1)(B + 1)
V real	20 de 32	16 de 24
Factor de ramificación máximo	Número de carreteras salientes (grado de salida)	6 acciones
Crecimiento	Exponencial en k	Polinomial: O(A · B)
Dificultad del problema general	NP-difícil (variante del viajante con visitas obligatorias)	Polinomial, se resuelve con teoría de números

Ruta de reparto. El espacio de estados crece como n · 2^k, porque el conjunto de entregados es un subconjunto de los clientes. Con 3 clientes el grafo es diminuto, pero con 20 clientes ya habría del orden de n · 10^6 estados. Esto refleja que el problema general es una variante del problema del viajante, de modo que las búsquedas exactas no escalan y en la práctica se recurre a heurísticas (A* con una heurística admisible, como el árbol de expansión mínima de los clientes pendientes) o a metaheurísticas.

Dosificación de líquidos. El número de estados está acotado por (A + 1)(B + 1), por lo que el crecimiento es polinomial en las capacidades. Con capacidades coprimas se alcanzan 2(A + B) estados, todos en la frontera del rectángulo de estados. Aquí la búsqueda exhaustiva es perfectamente viable aun con capacidades grandes.

7.5 Observaciones empíricas

En la ruta de reparto UCS expandió 12 nodos contra 13 de BFS; la diferencia es pequeña porque el grafo es pequeño y la solución más corta en acciones resultó ser también la más barata. En la dosificación ambos expandieron 14 de los 16 estados, ya que la meta está lejos del inicio y casi todo el espacio se explora antes de encontrarla. En grafos más grandes y con costos heterogéneos, BFS y UCS pueden devolver soluciones distintas, y solo UCS garantiza el costo mínimo.

8. Conclusiones

- La formulación con estado, acciones, costo y meta permite tratar problemas muy distintos con el mismo código: basta heredar de `ProblemaBusqueda` e implementar cuatro métodos.
- Representar el espacio de estados como `DiGraph` ponderado de NetworkX facilita calcular métricas, verificar resultados con algoritmos de la biblioteca y generar diagramas.
- El tamaño del espacio de estados es el factor determinante del costo de la búsqueda: en la dosificación crece de forma polinomial, mientras que en la ruta de reparto crece de forma exponencial con el número de clientes.
- Para problemas de mayor tamaño convendría generar el grafo de forma perezosa (solo los estados que se expanden) y usar búsqueda informada con heurísticas admisibles.

9. Referencias

- Russell, S. y Norvig, P. *Artificial Intelligence: A Modern Approach*. Capítulo sobre resolución de problemas mediante búsqueda.
- Documentación de NetworkX: https://networkx.org/documentation/stable/
- Cormen, T. et al. *Introduction to Algorithms*. Capítulos sobre algoritmos de caminos más cortos.
- Anthropic. (2026). Claude [Modelo de lenguaje grande]. https://claude.ai/